

• RESEARCH DECK

Deep Agents 첫 걸음

4주짜리 스터디의 공통 지도 한 장 — Overview ·
Quickstart · Customization

발표자	채널	일시
송태영	AI Odyssey	2026년 5월 5일

Vanilla 에이전트가 깨지는 세 지점

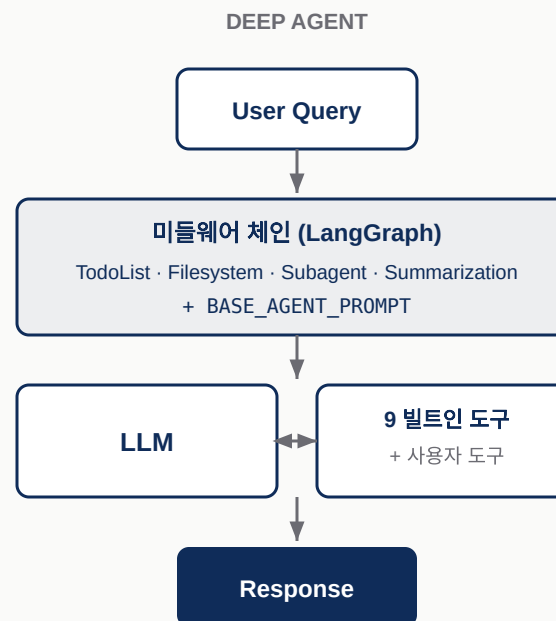
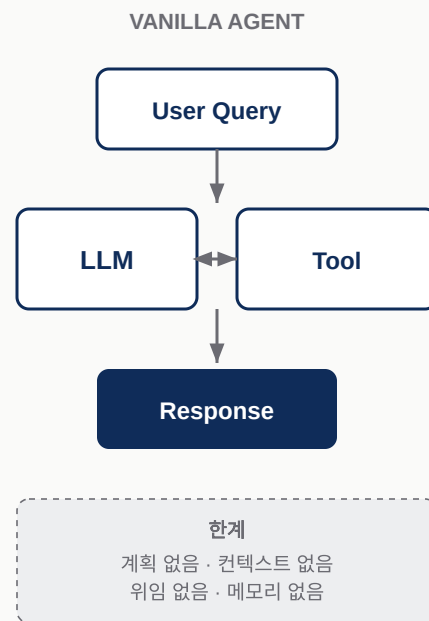
「LLM 한 번 + 도구 몇 개 + while 루프」 — 짧은 작업엔 충분하지만 단계가 많아지면 무너진다

깨지는 지점	무엇이 일어나는가
컨텍스트 오버플로	검색 결과·문서·이력이 누적돼 컨텍스트 윈도우를 잡아먹는다
계획의 부재	모델이 "다음 단계" 를 매 턴 즉흥으로 결정 — 길을 잃는다
위임 불가	한 모델이 모든 컨텍스트를 들고 있어 무거운 하위 작업이 메인을 오염시킨다

5단계까지는 견디지만 50단계에서는 무너진다 — 외부화·명시화·격리할 자리가 없기 때문.

같은 입력, 두 흐름

Vanilla vs Deep Agent — 미들웨어 한 커가 만드는 차이



미들웨어 체인이 매 turn (1) 계획 갱신, (2) 컨텍스트 오프로드, (3) 위임을 자동으로 끼워 넣는다.

폐쇄 시스템 셋이 같은 답을 발견

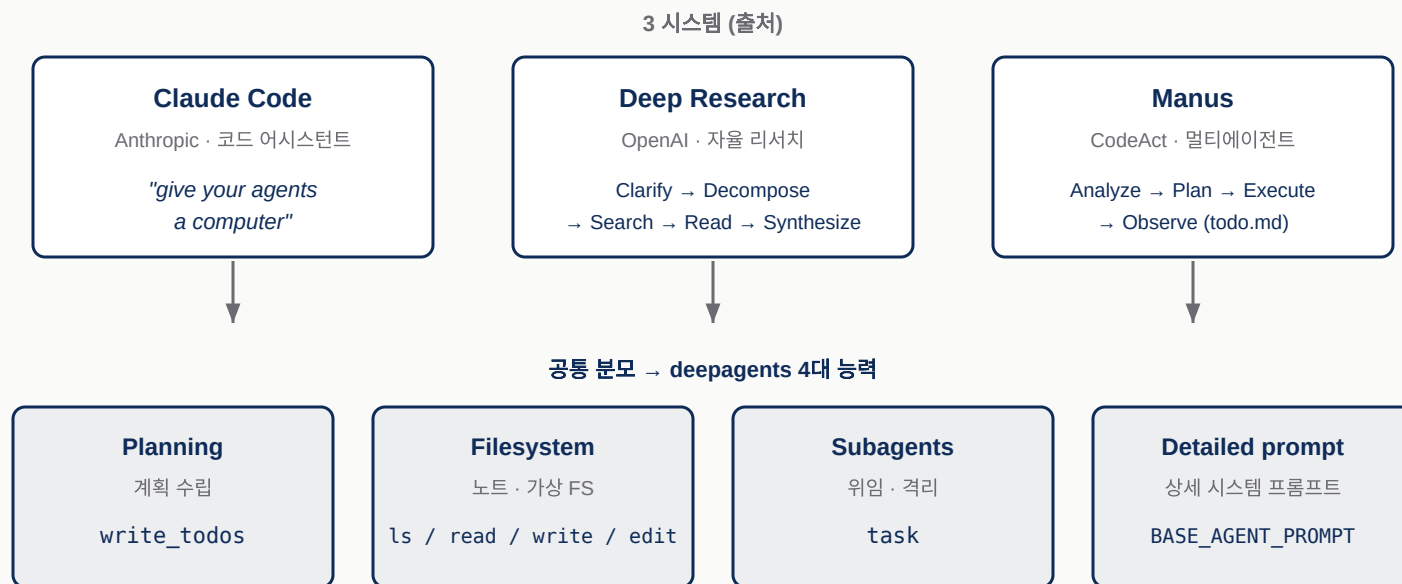
Claude Code · OpenAI Deep Research · Manus — 출시 주체도 구현도 다른데 같은 패턴

시스템	핵심 메커니즘
Claude Code (Anthropic)	파일시스템 · 서브에이전트 · 컴팩션 — <i>"give your agents a computer"</i>
OpenAI Deep Research	5단계: Clarify → Decompose → Search → Read → Synthesis. 한 작업당 검색 30~60회
Manus	CodeAct (Python 스크립트) + <code>todo.md</code> + 격리된 서브에이전트

셋 모두 **계획 파일 + 가상 파일시스템 + 서브에이전트** 라는 같은 세 축으로 수렴.

네 축으로의 수렴

Planning · Filesystem · Subagents · Detailed Prompt — 라이브러리화의 청사진



Harrison Chase 의 정리 — "Deep agent 란 계획(planning)을 수행하고, 서브에이전트(sub agents)를 사용하며, 파일시스템(file system)에 접근할 수 있고, 잘 다듬어진 프롬프트(detailed prompt)를 가진 에이전트다."

deepagents 의 스택 위치

LangGraph 그대로 깔고 그 위에 미들웨어 한 켠 — 새 프레임워크가 아니다



create_deep_agent() 의 반환은 LangGraph CompiledStateGraph — 기존 LangSmith · Platform · 체크포인트 · Store 가 그대로 재 활용된다.

비서의 4가지 — 할 일·노트·인턴·일기장

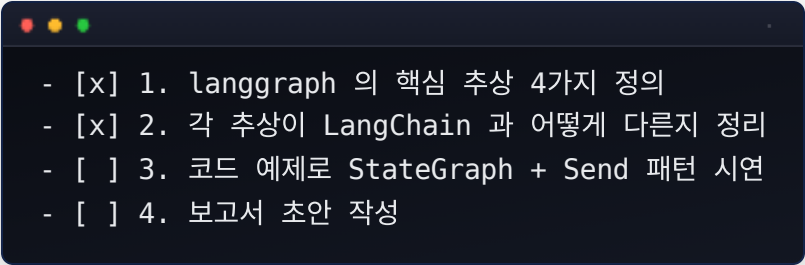
잘 갖춰진 비서에게 4종의 도구를 쥐여 준다



각 능력은 독립된 미들웨어 — 부분 사용·확장이 자연스럽고, `backend=` / `subagents=` / `middleware=` 로 조정한다.

Planning — `write\_todos` 의 no-op 트릭

모델이 자기 계획을 파일로 적어놓고 그 파일을 보며 일한다



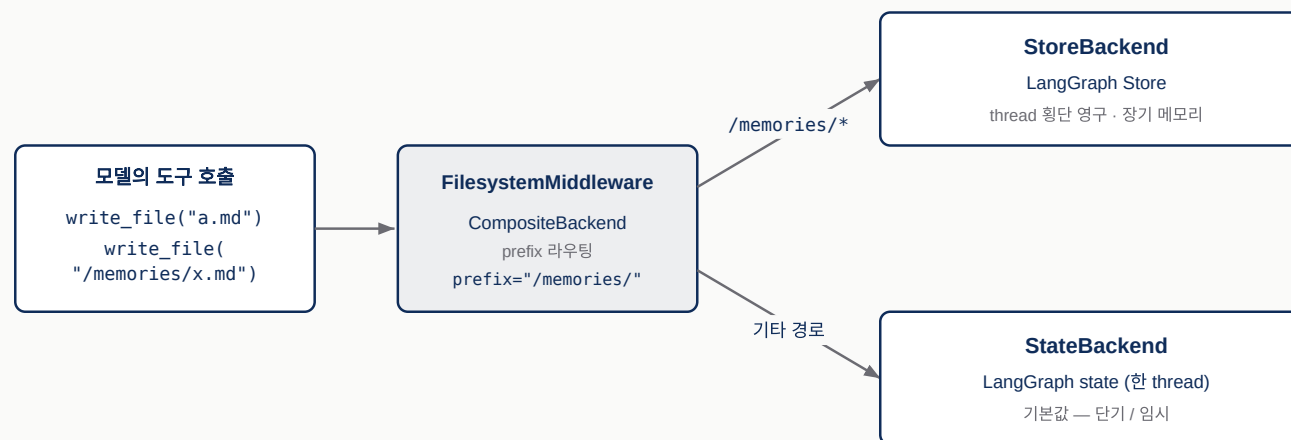
```
- [x] 1. langgraph 의 핵심 추상 4가지 정의  
- [x] 2. 각 추상이 LangChain 과 어떻게 다른지 정리  
- [ ] 3. 코드 예제로 StateGraph + Send 패턴 시연  
- [ ] 4. 보고서 초안 작성
```

- **호출 효과** — state `todos` 슬롯에 텍스트만 박힘. 시스템 어디에도 물리적 변화 없음
- **다음 turn** — 그 `todos` 가 시스템 프롬프트에 자동 재합성
- **결과** — 매 turn 자기 계획을 다시 마주 보는 모델

도구 자체는 **no-op** — 다음 turn 시스템 프롬프트에 todo 가 다시 들어가 모델이 계획을 잊지 않게 하는 컨텍스트 엔지니어링 트릭.

Filesystem — 같은 도구

한 벌의 파일 도구는 그대로 두고, 경로 prefix 만으로 임시 state 와 영속 Store 가 갈린다 — 모델은 분기를 모른다

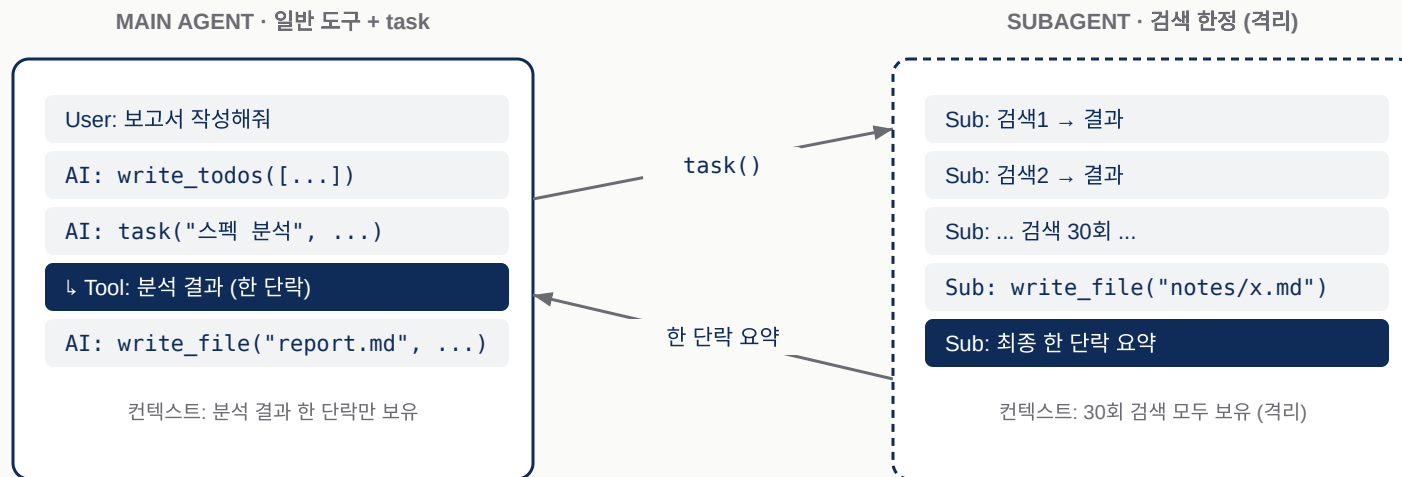


- **StateBackend** — 한 thread (디폴트, 임시)
- **StoreBackend** — 모든 thread (영속, 장기 메모리)
- **CompositeBackend** — prefix 라우팅 (`/memories/` → Store)

추상의 분리 — 도구 시그니처는 모델용으로 고정, 저장소 정책은 backend 한 줄로 결정 — Unix mount table 과 같은 발상.

Subagents — `task` 한 줄로 컨텍스트 격리

무거운 하위 작업은 격리된 서브에이전트에 위임 — 메인 컨텍스트가 깨끗해진다



- **컨텍스트 격리** — 자체 메시지 히스토리, 30회 검색의 turn 기록이 메인에 안 들어옴
- **도구 풀 격리** — `tools=` 필드로 메인보다 좁은 도구 풀 (예: 코딩 서버만 `execute`)
- **경로 규칙 격리** — `permissions=` 로 filesystem 도구의 read/write 경로 분리

위임 결정이 잘못돼도 셀 도구가 메인 컨텍스트로 돌아오지 않는다 — 보안·감사 측면의 분리.

장기 메모리 — thread 를 넘어 살아남는 파일

Checkpointing 는 한 thread, Store 는 모든 thread — `/memories/` prefix 가 후자로 라우팅

```
PYTHON

# Thread 1 - /memories/user.txt 에 사용자 이름 저장
config_1 = {"configurable": {"thread_id": "thread-1"}}
agent.invoke(
    {"messages": [("user", "Save my name 'Alice' to /memories/user.txt)]},
    config_1,
)

# Thread 2 - 다른 thread 인데도 같은 파일이 보인다
config_2 = {"configurable": {"thread_id": "thread-2"}}
agent.invoke(
    {"messages": [("user", "What is my name?)]},
    config_2,
)

# → 'Alice' 회신 (StateBackend 만이었으면 빈 파일시스템)
```

두 호출의 `thread_id` 가 다른데도 두 번째가 첫 번째의 `/memories/user.txt` 를 읽는다 — `CompositeBackend` 가 그 prefix 만 Store 로 라우팅하기 때문.

Quickstart 핵심 5줄

코어는 정말로 다섯 줄 — 빌트인 도구 9종이 자동으로 매달린다

```
PYTHON
from langchain.chat_models import init_chat_model
from deepagents import create_deep_agent
from tavily import TavilyClient
tavily = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])

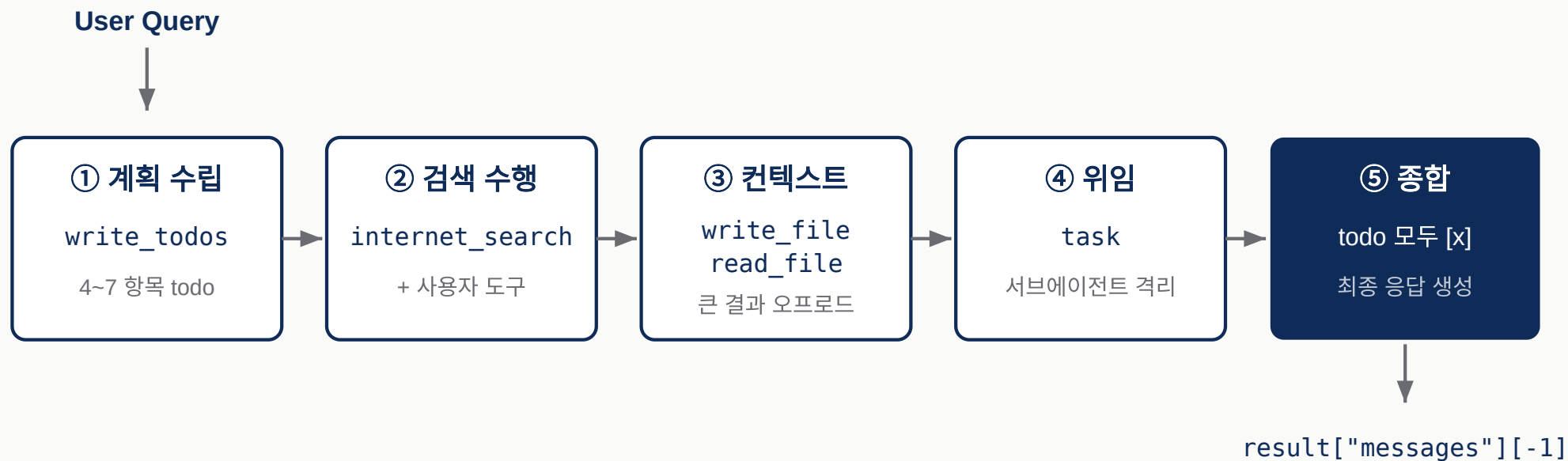
def internet_search(query: str, max_results: int = 5) -> dict:
    """Run a web search"""
    return tavily.search(query, max_results=max_results)

agent = create_deep_agent(
    model=init_chat_model("openai:gpt-4o-mini"),
    tools=[internet_search],
    system_prompt="You are an expert researcher.",
)
```

반환은 LangGraph `CompiledStateGraph` — `.invoke` / `.stream` / `.ainvoke` 모두 동작. 빌트인 9종 자동 매달림: `write_todos` · `ls` · `read_file` · `write_file` · `edit_file` · `glob` · `grep` · `task` · `execute` (sandbox 백엔드 필요).

`agent.invoke()` 백그라운드 5단계

한 줄 호출 뒤에서 일어나는 일 — todo → tool → file → task → synth



세 슬롯이 모두 채워진 모습이 4대 능력 발동의 흔적 — `result["files"]` (FilesystemMiddleware), `result["todos"]` (TodoListMiddleware), `result["messages"]` (최종 응답).

미들웨어 체인 7층 — 모델은 모른다

모델은 도구가 거기 있다고 인식할 뿐, 미들웨어 자체는 보이지 않는다

- 1 **TodoListMiddleware** — 계획 갱신
- 2 **SkillsMiddleware** — `skills=` 인자가 있을 때
- 3 **FileSystemMiddleware** — 오프로드 / 로드
- 4 **SubAgentMiddleware** — 선언형 서브에이전트
- 5 **SummarizationMiddleware** — 필요 시 컨텍스트 압축
- 6 **PatchToolCallsMiddleware**
- 7 **AsyncSubAgentMiddleware** — async 서브에이전트

이 투명성이 deepagents 가 LangGraph 의 "직접 짠 워크플로" 와 다른 결정적 지점.

청사진 — Core + Features

17개 파라미터를 세 묶음으로 — 첫 90%는 Core 셋만



왼쪽 3개를 합성하여 LangGraph 체인을 빌드 · 오른쪽 3개로 행동을 확장

- **Core (3)** — `model`, `system_prompt`, `tools` (가장 자주)
- **Features (5)** — `backend`, `subagents`, `interrupt_on`, ... (영속·위임)
- **Advanced (9)** — `middleware`, `skills`, `response_format`, ... (정밀 제어)

Model 바꾸기 — 문자열 vs 객체

길 1은 빠르게, 길 2는 정확하게

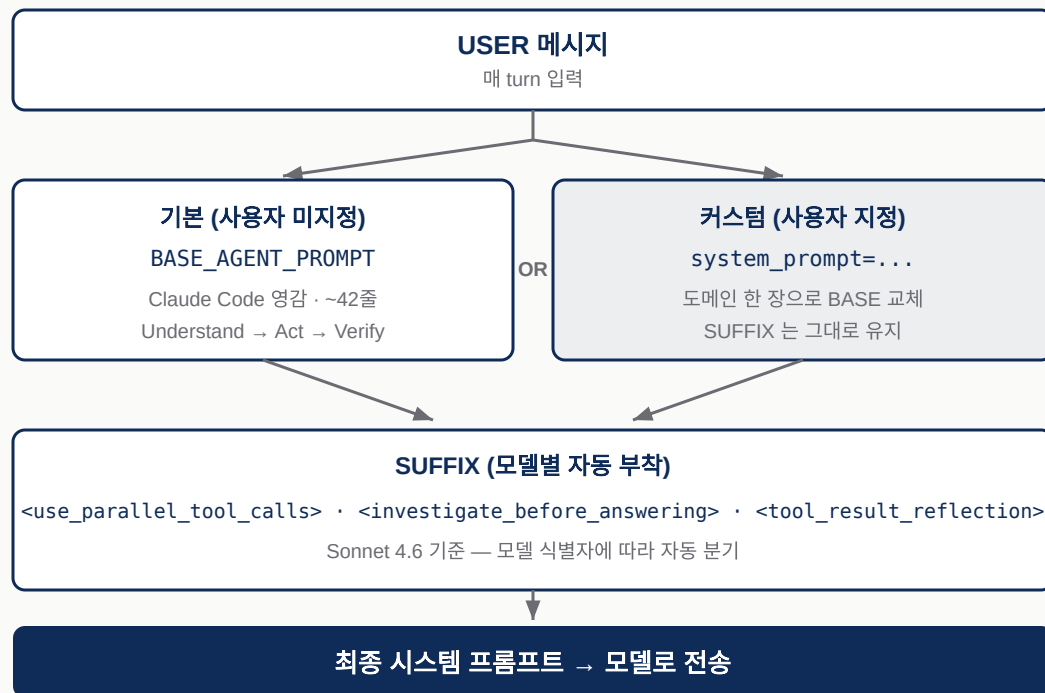
```
PYTHON
# 길 1 - provider:model 문자열 (디폴트 경로)
model = init_chat_model("openai:gpt-4o-mini")
agent = create_deep_agent(model=model)
```

```
PYTHON
# 길 2 - LangChain 객체 (Ollama / Bedrock / temperature 같은 디테일)
model = init_chat_model(
    model="llama3.1",
    model_provider="ollama",
    temperature=0,
)
agent = create_deep_agent(model=model)
```

`OPENAI_BASE_URL` 만 같아 끼우면 OpenAI · 사내 프록시 · Bedrock 호환 프록시 모두에서 동일 코드가 작동.

시스템 프롬프트 3단 합성

사용자 한 장은 BASE 를 교체하지 않는다 — 그 앞 (USER 자리) 에 prepend 된다



BASE 의 핵심 지침: *"NEVER add unnecessary preamble"* (불필요한 서두는 절대 붙이지 말 것) · **Understand → Act → Verify** 3단 워크

플로 USER → BASE (또는 CUSTOM) → SUFFIX . 사용자 한 장은 USER 자리에 prepend — BASE 와 모델별 SUFFIX 가 살아남는다.

Deep agent 는 언제? — 세 갈래길의 결정

"단계 5+ · 컨텍스트 윈도우 절반+ · 위임 가능" 셋이 동시에 참이면 deep agent

상황	추천	이유
짧은 Q&A, 단일 도구 호출	<code>create_agent</code>	미들웨어 오버헤드 없이 가볍게
복잡한 다단계 + 큰 컨텍스트 + 위임	<code>create_deep_agent</code>	4대 능력 자동 ON, BASE 프롬프트 검증됨
흐름 결정적이고 노드 통제 필요	LangGraph workflow	StateGraph 로 명시 모델링
짧은 작업 + 도구 정책 까다로움	<code>create_agent</code> + 미들웨어 직접 조립	<code>create_deep_agent</code> 의 부분집합

셋 중 하나만 참이면 다른 길을 검토 — deep agent 는 만능이 아니다.

Q&A

감사합니다

지도 한 장, 같이 들고 갑니다

발표자

송태영 · Deep Agents 첫 걸음

다른 주제

Why · 4대 능력 · Quickstart
청사진 · 결정

자료

scripts/01~04 · figs/01~09
99_references.md